

# AI AGENTS

## MCP | LANGCHAIN | REACT

BUILD INTELLIGENT, REAL-WORLD AI APPLICATIONS



WRITTEN BY

ER NIMISH MISHRA

# AI Agents, MCP, LangChain & React

A practical, developer-oriented guide to building connected, tool-using AI applications.

## DESCRIPTION

This guide introduces the concepts and implementation patterns behind modern AI agents. It explains why language models need access to external tools and application data, and how agent frameworks coordinate model reasoning, tool selection, execution, and responses.

The LangChain sections cover chat models, message types, prompt templates, output parsers, Runnable composition, tools, tool calling, agent creation, conversation memory, and middleware. The material also discusses the distinction between a chat model and an agent, along with sequential and parallel workflow patterns.

The MCP sections explore the Model Context Protocol as a standardized connection between AI applications and external systems. Topics include MCP servers and clients, tools, resources, prompts, JSON-RPC 2.0 messaging, request/response patterns, notifications, batching, transport options, lifecycle stages, sessions, and multi-server tool access.

The final sections address practical implementation concerns such as error handling, authentication, role- and scope-based tool access, and structured data validation with Pydantic. Examples and flow diagrams throughout the guide are intended to help connect the concepts to application development.

## FOCUS AREAS

Agent architecture • Tool orchestration • MCP integration  
Workflow composition • Security • Data validation

Written by ER Nimish Mishra

# Index

Section references point to the page numbers in this edition.

|    |                                                          |       |
|----|----------------------------------------------------------|-------|
| 01 | Why AI Agents Are Required                               | 4     |
| 02 | Why LangChain Is Used; Advantages & Limitations          | 4–8   |
| 03 | Chat Models, Messages, Invoke & Model Providers          | 9–15  |
| 04 | LCEL, Runnable, Sequential & Parallel Workflows          | 16–20 |
| 05 | Model Types, Prompt Templates & Output Parsers           | 21–26 |
| 06 | Structured Output; Components vs. Runnables              | 26–28 |
| 07 | Tools, Tool Schemas, Binding & Tool-Calling Loop         | 27–34 |
| 08 | Agent Creation, Memory & Conversation State              | 35–37 |
| 09 | Agent Workflow Example & Middleware                      | 37–41 |
| 10 | MCP Server Fundamentals & Why MCP Is Useful              | 42–45 |
| 11 | MCP Tools, Resources & Prompts                           | 45–55 |
| 12 | JSON-RPC 2.0: Requests, Responses & Errors               | 46–50 |
| 13 | MCP Tool Calls, Resource Operations & Notifications      | 50–55 |
| 14 | Prompt Retrieval, Batching & JSON-RPC Benefits           | 54–57 |
| 15 | MCP Transport: STDIO, HTTP & Server-Sent Events          | 58–61 |
| 16 | MCP Lifecycle: Initialization, Operation & Shutdown      | 60–66 |
| 17 | MCP Client Discovery, Tool Execution & Sessions          | 67–69 |
| 18 | Notifications, Multi-Server Tool Access & Error Handling | 68–71 |
| 19 | MCP Authentication, JWT & Scope-Based Tool Security      | 71–72 |
| 20 | Pydantic & Structured Data Validation                    | 73–74 |

The page references include the added opening pages. The original instructional pages follow in their existing order.

## Why Are Agents Required?

Agentic AI came into the picture to make applications smarter and more capable by using AI models such as **OpenAI**, **Anthropic**, and other **LLMs**.

An LLM has limited knowledge and context. It cannot automatically access your application's current data, databases, APIs, or external systems.

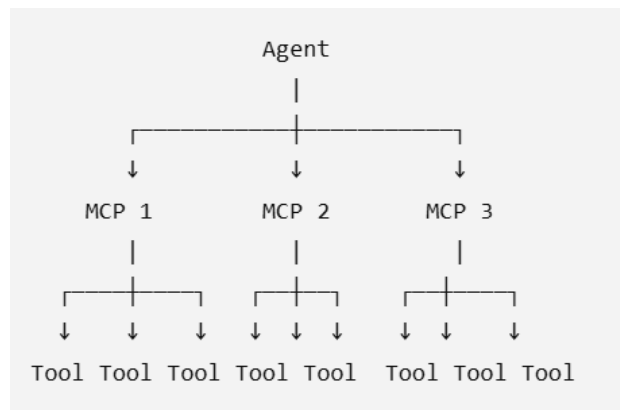
Agents act as a bridge between the LLM and the real world.

They can:

- Fetch the latest/current data from databases and APIs.
- Use tools to perform actions.
- Decide which tool or data source is required.
- Execute multi-step tasks based on the user's request.
- Pass the relevant information back to the LLM to generate the final response.

Agent can use multiple MCPs

An agent can connect to multiple **MCP servers**, and each **MCP server** can expose multiple tools.



## Why is LangChain Required?

When we use an LLM provider's SDK directly, we often need to handle many things manually, such as API calls, request/response formats, prompt management, tool calling, message structures, and connecting multiple components.

**LangChain** makes this process easier by providing a framework and ready-made abstractions. Instead of writing everything manually, LangChain provides components that handle much of the integration work for us.

For example, without LangChain:

```
api_key=os.getenv("OPENAI_API_KEY")

url = "https://api.openai.com/v1/chat/completions"

headers = {
    "Content-Type": "application/json",
    "Authorization": f"Bearer {api_key}"
}
payload = {
    "model": "gpt-4o-mini",
    "messages": [ {"role": "user", "content": "What is AI in short" } ]
}
```

Here everything happening manually but with LangChain:

```
model = ChatOpenAI(model="gpt-4o-mini")

messages = [SystemMessage(content="You are a python trainer. Answer in three words"),
            HumanMessage(content="tell me about python ?")]

response = model.invoke(messages)
```

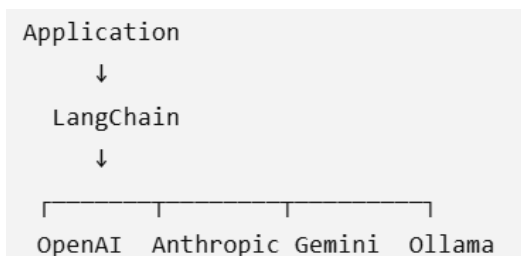
LangChain provides integrations for different LLM providers, prompts, tools, agents, output parsers, memory, and other components.

### Advantages of LangChain

LangChain is useful because it provides a higher-level framework for building LLM applications, so you don't have to implement everything manually.

### Advantages of Lang chain :

1. **Easy LLM Integration** : You can work with different LLM providers using a common interface.



This makes it easier to change models without rewriting the entire application.

2. **Prompt Management** : LangChain provides reusable prompt templates instead of creating prompts manually every time.

```
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_template(
    "Explain {topic} in simple words."
)
```

Then you can invoke

```
prompt.invoke({"topic": "Python"})
```

### 3. Easy Tool Integration

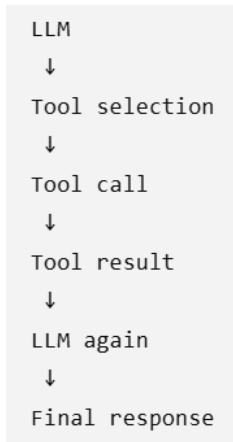
You can define functions as tools and allow an agent to decide when to use them.

```
@tool
def get_weather(city: str):
    ...
```

The agent can then decide:

```
User
↓
Agent
↓
Do I need weather information?
↓
YES
↓
get_weather()
↓
Result
↓
LLM
↓
Answer
```

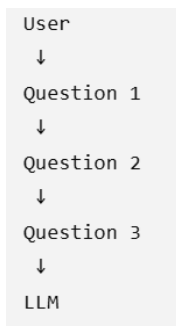
4. **Agent Development** : Building an agent manually requires handling things such as:



LangChain provides abstractions to simplify this process.

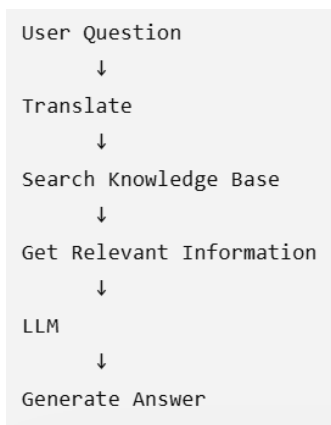
#### 5. **Memory / Conversation Management**

For conversational applications, LangChain provides components for managing messages and conversation state.



The application can provide the relevant conversation history to the model.

6. **Chains and Workflows** : You can connect multiple operations together.

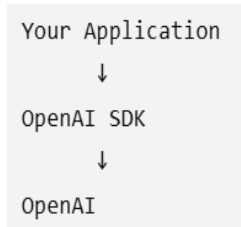


Each step can be represented as part of a larger workflow.

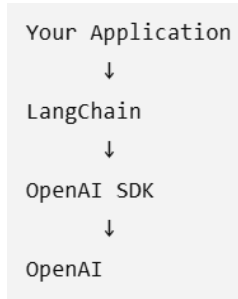
## Disadvantages of langChain:

### 1. Adds an Extra Abstraction Layer :

Instead of directly calling the LLM SDK:



with LangChain:



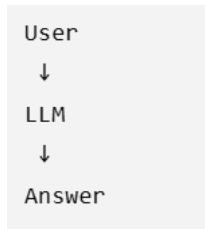
This additional layer can make debugging and understanding the underlying API more difficult.

### 2. More Complexity for Simple Applications

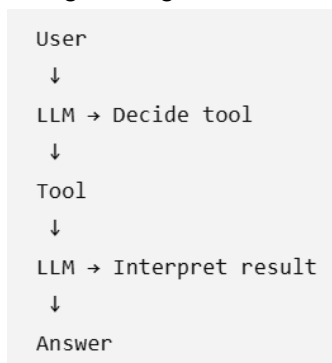
### 3. Agent Can Increase Cost

This is especially important with agents.

A simple request might require:



An agent might do:



## 1. Chat Models in LangChain

A Chat Model is the LangChain interface used to communicate with modern conversational LLMs such as OpenAI, Anthropic, Google Gemini, Mistral, and others.

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-4o-mini"
)

response = model.invoke(
    "What is term insurance?"
)

print(response.content)
```

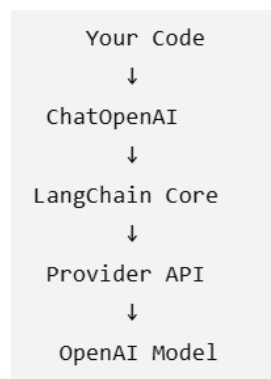
Here, **ChatOpenAI** is the chat model interface, while gpt-4o-mini is the actual model.

- **Chat Model Architecture:**

Suppose you write:

```
model = ChatOpenAI(
    model="gpt-4o-mini"
)
```

Your application doesn't directly implement the OpenAI API protocol. LangChain provides an abstraction:



- **Different Providers:**

One of the useful things about LangChain is that the application architecture can remain similar while the underlying provider changes.

Ex: **ChatOpenAI()**, **ChatAnthropic()** etc

- **Invoke() :**

The most basic operation. It's basically Send an input to the model and wait for the result.

```
response = model.invoke(  
    "What is the difference between term insurance and whole life insurance?"  
)  
  
print(response)
```

The returned object isn't necessarily just a string. It is usually an `AIMessage`. You commonly access:

```
response.content
```

- **AIMessage :**

When you call a chat model it returns a response that is called `AIMessage`, this message consists with :

```
AIMessage  
|  
+-- content  
+-- response metadata  
+-- usage metadata  
+-- additional information
```

But the message can also contain metadata such as token usage or provider-specific information.

This is why LangChain doesn't simply return a plain string.

- **The Four Important Message Types:**

When working with chat models, you should know these:

```
SystemMessage  
HumanMessage  
AIMessage  
ToolMessage
```

1. **System message** are the instructions for the model ex :system prompt
2. **Human message** is the user's message, the input one
3. **AI message** is response from model

4. **Tool message** is the result returned by a tool  
For example:

```
AI → Call get_premium()  
Tool → ₹25,000  
AI → Your premium is ₹25,000.
```

- System + Human + AI :

A real conversation can look like:

```
System:  
You are an insurance assistant.  
  
Human:  
What is my premium?  
  
AI:  
Your premium is ₹25,000.  
  
Human:  
When is it due?  
  
AI:  
Your next premium is due...
```

But in lang chain it looks like :

```
messages = [  
    SystemMessage(  
        content="You are an insurance assistant."  
    ),  
  
    HumanMessage(  
        content="What is my premium?"  
    ),  
  
    AIMessage(  
        content="Your premium is ₹25,000."  
    ),  
  
    HumanMessage(  
        content="When is it due?"  
    )  
]
```



The model can use the conversation history to understand:  
"it" = the premium using the previous conversation.

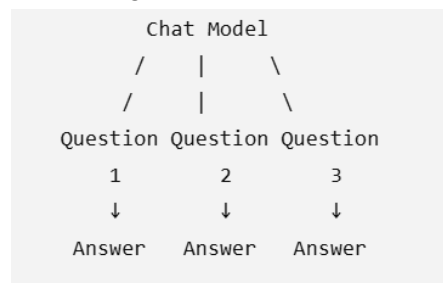
- **Chat Models and Batch** : let say you've multiple queries like

```
questions = [  
    "What is premium?",  
    "What is sum assured?",  
    "What are riders?"  
]
```

In that you can use

```
responses = model.batch(questions)
```

How this get executed inside the model



This is useful when requests are independent.

- **Async Chat Models**

In an async application, such as FastAPI, you generally don't want to block the event loop unnecessarily.

```
response = await model.ainvoke(  
    "What is term insurance?"  
)
```

- **Chat Models + Prompt Templates**

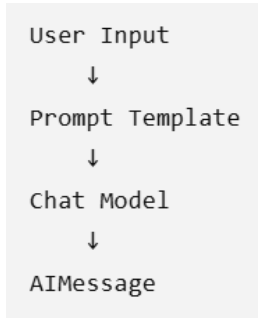
A very common LangChain pattern is:

```
from langchain_core.prompts import ChatPromptTemplate  
  
prompt = ChatPromptTemplate.from_messages([  
    ("system", "You are an insurance expert."),  
    ("human", "{question}")  
)  
  
chain = prompt | model
```

Then

```
response = chain.invoke({
    "question": "What is term insurance?"
})
```

The architecture becomes:



So how this architecture differs from traditional architecture:

| Feature           | Standard String Prompts        | Chat Model + Prompt Templates                                                                  |
|-------------------|--------------------------------|------------------------------------------------------------------------------------------------|
| Structure         | Raw single string              | Role-based message arrays ( <code>System</code> , <code>User</code> , <code>Assistant</code> ) |
| Reusability       | Manual string formatting       | Parameterized templates with validated input schema                                            |
| State/Memory      | Appending text manually        | Dynamic injection of message arrays into templates                                             |
| Model Flexibility | Tied to legacy completion APIs | Swap underlying providers (OpenAI, Anthropic, Gemini) seamlessly                               |

● **Chat Models + Tools :**

This is where chat models become the foundation of agents.  
Suppose you have:

```
get_premium()
get_policy_status()
get_policy_details()
```

The chat model can be given access to those tools.

Then:

```
User
↓
Chat Model
↓
"I need premium information"
↓
Tool Call
↓
get_premium()
↓
Database/API
↓
Tool Result
↓
Chat Model
↓
Final Answer
```

So the chat model isn't necessarily doing the database lookup itself.

It decides: "I need this tool."

Your application executes the tool.

- **The Core Methods You Should Remember**

For LangChain chat models, remember these first:

```
model.invoke()
model.ainvoke()

model.stream()
model.astream()

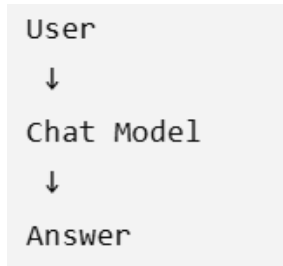
model.batch()
model.abatch()
```

- **Chat Model vs Agent**

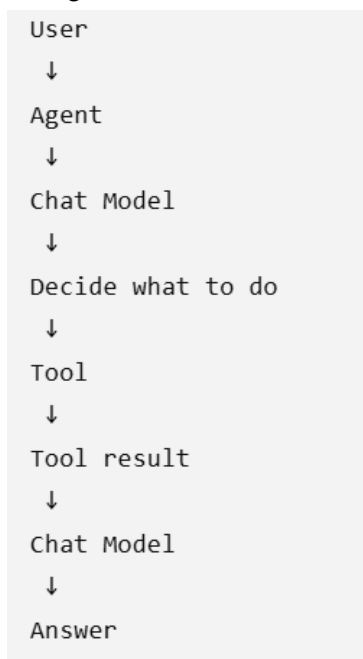
This distinction is extremely important.

The Chat Model is the brain/model interface. An Agent is a larger system that uses a chat model plus tools and decision-making logic.

#### A chat Model

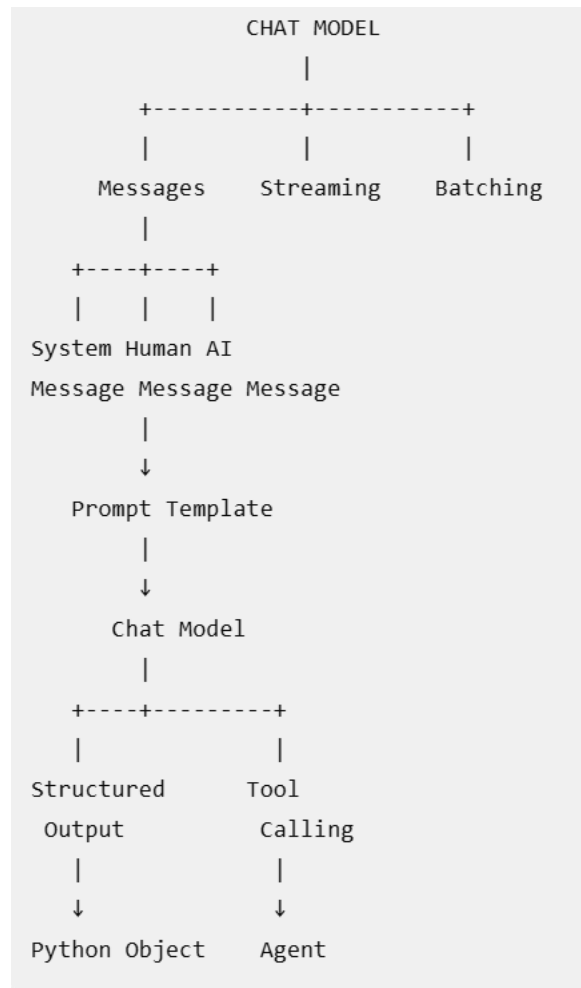


#### An Agent



- **The mental model to remember**

If you're studying LangChain, remember this:



## 2. **LCEL Runnable**

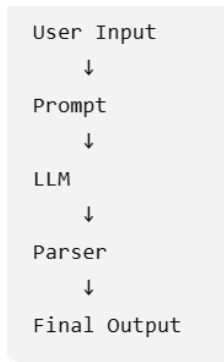
LCEL = LangChain Expression Language

LCEL lets you connect LangChain components together using a common Runnable interface.

Think of a Runnable as:

Something that takes an input and produces an output.

For example:



Each component can be treated as a Runnable.

A very common LCEL syntax is:

```
chain = prompt | model | parser
```

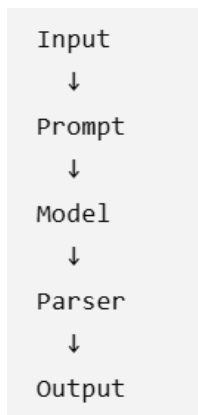
Here “ | “ means:

Send the output of the left component into the right component, it's called Runnable Pipe, The pipe is used to create a sequential chain.

the execution of this chain

chain = prompt | model | parser

will look like



Why is this useful?

Instead of manually doing:

```
prompt_value = prompt.invoke(data)
model_response = model.invoke(prompt_value)
result = parser.invoke(model_response)
```

you can simply write:

```
chain = prompt | model | parser
```

A LangChain chain using `|` executes each Runnable sequentially (first prompt get executed then model then parser), where the output of one component becomes the input to the next component.

When we have multiple chains, the output of the first chain can be used as the input for the next chain. Instead of executing each chain manually and passing the output between them, LangChain allows us to connect them using the pipe (`|`) operator.

This makes the code cleaner, simpler, and easier to compose.

### 3. **RunnableParallel:**

Take the same input and send it to multiple things at the same time.

Normal sequential flow

Suppose you have:

```
input
↓
step_1
↓
step_2
↓
step_3
```

In LangChain LCEL

```
chain = step_1 | step_2 | step_3
```

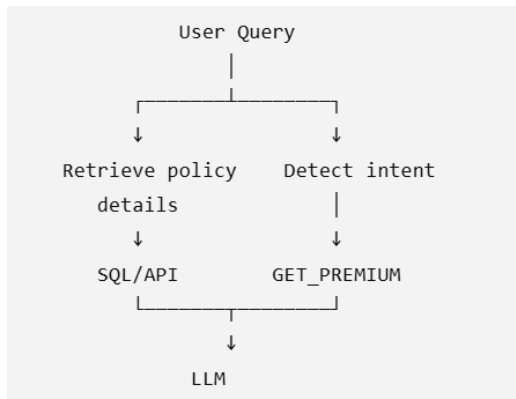
If you implement runnable parallel here then the flow will look like this

```
input ———┐
           ├──> step_1
           └──> step_2
```

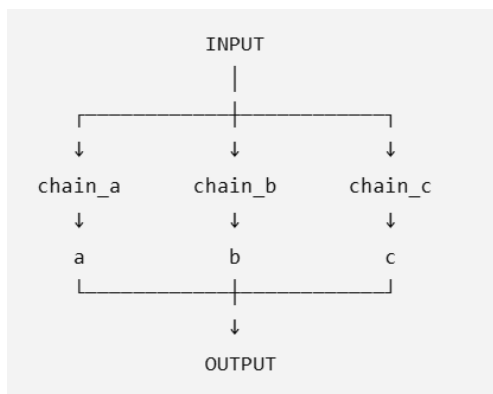
Both step\_1 and step\_2 receive the same input and can execute concurrently.

```
from langchain_core.runnables import RunnableParallel

chain = RunnableParallel(
    result1=step_1,
    result2=step_2
)
```



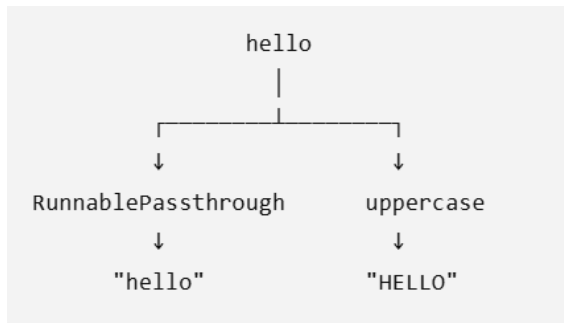
The most important thing to understand is that `RunnableParallel` does NOT mean "Run the entire chain in parallel." It means **"Run these branches in parallel using the same input."**



#### 4. **RunnablePassthrough :**

It doesn't change the input. Just pass it forward.

```
RunnableParallel(
    original=RunnablePassthrough(),
    uppercase=RunnableLambda(lambda x: x.upper())
)
```



Why not just use the input directly?

Because RunnableParallel expects each branch to be a runnable.

**The easiest way to remember :**

```
LCEL
├─ Pipe      → A → B → C
├─ Parallel  → A → (B + C)
└─ Passthrough → keep original input
```

```
Agent
└─ decides what action/tool to use
```

```
State
└─ where am I + what data do I currently have?
```

```
Memory
└─ what do I remember from before?
```

```
Document
└─ what piece of knowledge am I retrieving?
```

```
LangGraph
└─ how do I manage the entire stateful workflow?
```

## Types of Models

There are two broad categories of AI models:

### 1. Open Models

Examples: **Mistral, Qwen**

We generally refer to these as open models rather than open-source models.

An open model provides access to the model weights, but it does not necessarily provide the complete source code, training process, datasets, or details of how the model was developed and implemented

### 2. Closed Models

Examples: **GPT, Anthropic's Claude**

The model weights and implementation details are not publicly available.

Users typically access these models through APIs or hosted services.

## Prompt Template in LangChain

A Prompt Template in LangChain is a reusable template for creating prompts dynamically.

Instead of writing a prompt every time:

```
You are a tutor.  
Explain Python in 100 words.
```

you create a template with **variables**:

```
from langchain_core.prompts import ChatPromptTemplate  
  
prompt = ChatPromptTemplate.from_messages([  
    ("system", "You are a tutor. Answer in {limit} words."),  
    ("user", "{question}")  
])
```

Now you can provide different values:

```
result = prompt.invoke({  
    "limit": 100,  
    "question": "What is Python?"  
})
```

## Prompt Template + Model

This is where LangChain becomes particularly useful:

```
prompt = ChatPromptTemplate.from_messages([("system", "You're a tutor. Answer in {limit} words."),
    ("user", "{question}")
])

model = ChatOpenAI( model="gpt-4o-mini", temperature=0)

parser = StrOutputParser()

chain = prompt | model | parser

print(
    chain.invoke({
        "limit": 60,
        "question": "What is an API?"
    })
)
```

Types of Prompt Template :

In **LangChain**, there are several types of prompt templates, depending on how you want to structure the prompt.

```
Prompt Templates
|
├── PromptTemplate
|   └── Simple text-based prompt
|
├── ChatPromptTemplate
|   ├── System Message
|   ├── Human/User Message
|   ├── AI Message
|   └── MessagesPlaceholder
|       └── Chat History / Dynamic Messages
|
├── FewShotPromptTemplate
|   ├── Example 1
|   ├── Example 2
|   └── Final Input
|
└── FewShotChatMessagePromptTemplate
    ├── Example User Message
    ├── Example AI Message
    └── Final User Message
```

## Output parser :

In LangChain, an Output Parser is the component that takes the model's response and converts it into a format your application can easily use.

Why do we need it?

A chat model doesn't always return a plain Python string. For example, it may return:

```
AIMessage(  
    content="An API is a way for two applications to communicate."  
)
```

But your application might only want:

```
</> Python  
  
"An API is a way for two applications to communicate."
```

That's what StrOutputParser does.

## Common parsers to remember

```
Output Parsers  
|  
├─ StrOutputParser  
|   └─ AIMessage → String  
|  
├─ JsonOutputParser  
|   └─ Model response → JSON  
|  
├─ PydanticOutputParser  
|   └─ Model response → Pydantic object  
|  
└─ CommaSeparatedListOutputParser  
    └─ Model response → List
```

### 1. StrOutputParser

```
from langchain_core.output_parsers import StrOutputParser  
  
parser = StrOutputParser()  
  
chain = prompt | model | parser
```

## 2. JsonOutputParser

```
from langchain_core.output_parsers import JsonOutputParser

parser = JsonOutputParser()

chain = prompt | model | parser
```

## Runnable vs Component

This is a very important distinction in LangChain. The easiest way to understand it is:

**Component = what the thing does.**

**Runnable = how the thing can be executed/composed.**

### 1. Component :

A component is a building block that performs a particular job.

Examples:

```
Prompt Template → creates the prompt
Chat Model       → calls the LLM
Output Parser    → converts the output
Retriever        → retrieves documents
Tool             → performs an action
```

```
LangChain Components
|
├─ Prompt
├─ Model
├─ Parser
├─ Retriever
└─ Tool
```

### 2. Runnable

Runnable is an interface/abstraction that allows a component to be executed in a standard way.

The important methods include:

```
.invoke()
.batch()
.stream()
```

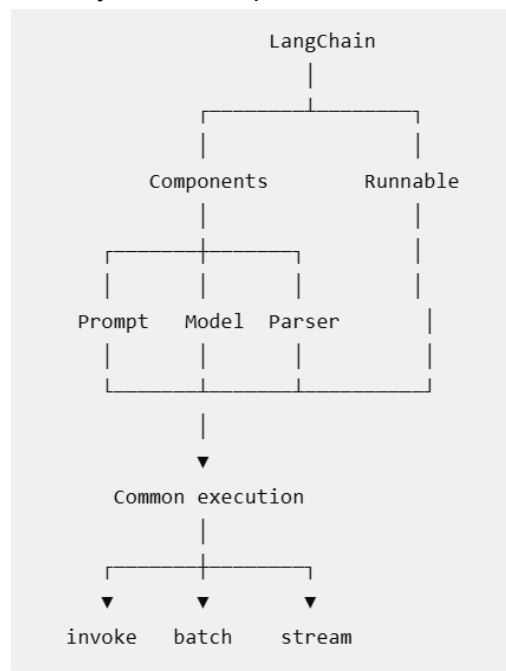
A very simple analogy, think about a USB interface. Different devices do different jobs:

```
Keyboard → input  
Mouse    → input  
Printer  → output  
Storage  → storage
```

But they can all follow a common USB interface.  
Similarly:

```
Prompt    → creates prompt  
Model     → generates response  
Parser    → parses response  
Retriever → retrieves data
```

The key relationship



**Component tells you what it does; Runnable tells you how you can run and compose it.**

## Structure output :

Structured Output means controlling the format/schema of the LLM's response so that your application can reliably consume it.

```
structured_model = model.with_structured_output(  
    {  
        "type": "object",  
        "properties": {  
            "intent": {  
                "type": "string",  
                "description": "The user's intent"  
            },  
            "confidence": {  
                "type": "number",  
                "description": "Confidence score between 0 and 1"  
            }  
        },  
        "required": ["intent", "confidence"]  
    }  
)  
  
response = structured_model.invoke(  
    "I want to check the status of my insurance policy"  
)
```

## Tools

In LangChain, a Tool is a function that an LLM/agent can call to perform an action that the model itself cannot reliably do. We have two types of tools

1. Built in tools
2. Custom tools

Think of it like:

**LLM** = brain that decides what to do

**Tool** = capability that actually does it

Without a tool, suppose the user asks:

"What is the weather in Mumbai?"

The LLM can answer based on its knowledge, but it doesn't automatically know the current weather.

but if You give a weather tool:

```
User
↓
LLM
↓
"I need current weather → call weather tool"
↓
Weather Tool
↓
Current weather data
↓
LLM
↓
Final answer
```

A tool is usually a **Python function with a defined name, description, and input schema** that the model can request to call.

```
from langchain_core.tools import tool

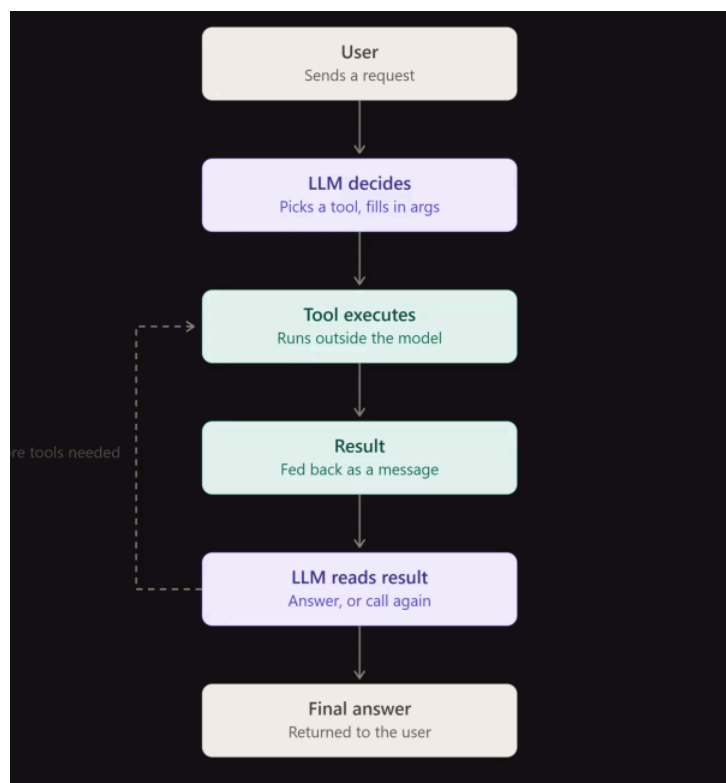
@tool
def add_numbers(a: int, b: int) -> int:
    """Add two numbers."""
    return a + b
```

Now you have a tool:

```
Tool
|
├─ Name: add_numbers
├─ Description: Add two numbers
├─ Input: a, b
└─ Function: a + b
```

### How the tool calling works :

This architecture just to call the tool not to execute the tool



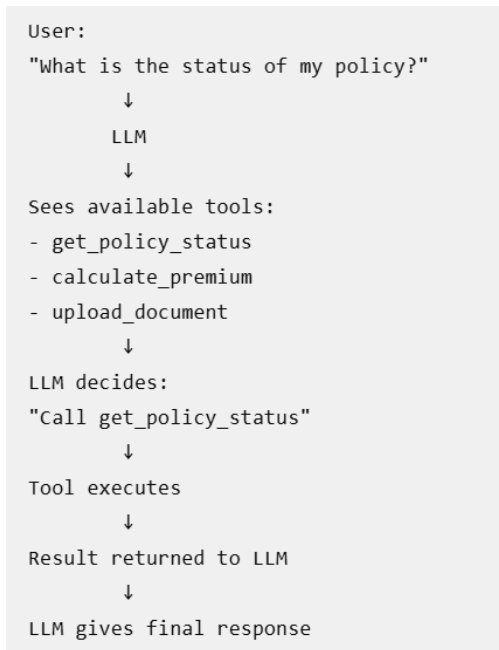
So tools are just functions ?

No not really, the important difference is that a Tool has **metadata/schema** that the LLM can **understand**.

So, once we build the tools, the next question is: **how will those tools be called?**

We need a medium or protocol to invoke them, and there are multiple protocols available for this, such as **MCP, REST, gRPC, and WebSockets**.

But the question is still in my mind, how did LLM come to know about the tool ?  
The LLM is given a list of available tools, and based on the user's request, the LLM decides which tool should be called and what arguments to pass to it.



**The LLM does not execute the tool. It only requests the tool call.** The how the tool get executes

- LLM = decides/request
- Application/Agent Runtime = executes
- Tool = performs the actual operation

**LLM to use a tool**, it generally needs these 3 core pieces of information:

1. **Name** — What is the tool called?
2. **Description** — What does the tool do and when should the LLM use it?
3. **Arguments** — What inputs does the tool require?

```
Tool: get_policy_details

Name:
get_policy_details

Description:
Fetch policy details using the customer's policy number.

Arguments:
{
  "policy_number": "string"
}
```

The flow is:

**LLM sees tool definition → understands when to use it → generates tool call with arguments → tool executes → result comes back to LLM**

```
model=ChatOpenAI(model="gpt-4o-mini")

@tool
def multiply(a: int, b: int) -> int:
    """Multiply two numbers and return the exact result."""
    return a * b

@tool
def word_count(text: str) -> int:
    """Count how many words are in a piece of text."""
    return len(text.split())

model_with_tools= model.bind_tools([
    multiply,word_count
])

response =model_with_tools.invoke("what is 90854 multiplied by 2345")
print(response)
```

In langchain using the `bind_tools` function you can bind all the tools with LLM, `bind_tools()` sends the tool definitions to the LLM as part of the model request. LangChain takes the Python functions and converts them into **tool schemas** roughly like this:

```
[
  {
    "name": "multiply",
    "description": "Multiply two numbers and return the exact result.",
    "parameters": {
      "type": "object",
      "properties": {
        "a": {"type": "integer"},
        "b": {"type": "integer"}
      },
      "required": ["a", "b"]
    }
  },
  {
    "name": "word_count",
    "description": "Count how many words are in a piece of text.",
    "parameters": {
      "type": "object",
      "properties": {
        "text": {"type": "string"}
      },
      "required": ["text"]
    }
  }
]
```



So when you do:

```
response = model_with_tools.invoke(
    "what is 90854 multiplied by 2345"
)
```

the LLM effectively receives:

```
User:
what is 90854 multiplied by 2345

Available tools:

multiply(a, b)
Description: Multiply two numbers and return the exact result.

word_count(text)
Description: Count how many words are in a piece of text.
```

The LLM doesn't know the Python implementation of the tool. It knows the tool's name, description, and argument schema, which are provided to it. It decides which tool to request and what arguments to provide. The application actually executes the tool.

Now till this point the tools has been called but the output from the tool has not came yet look at this response.content is empty why ?

```
26
27 response =model_with_tools.invoke("what is 90854 multiplied by 2345")
28 print("complete response from LLM")
29 print(response)
30 print(" ")
31 print("=====")
32 print(response.content)
33 print("=====")
34 print(response.tool_calls)
35
36 print("_____tool details _____")
37 if response.tool_calls:
38     tool_call = response.tool_calls[0]
39     print(tool_call["name"])
40     print(tool_call["args"])
41
42     tools = {
43         "multiply" : multiply,
```

Python 3.11.4

Problems Output Debug Console Terminal Ports

• (first app) PS C:\Users\nimishmishra\Desktop\Agentic AI\first app> & "c:\Users\nimishmishra\Desktop\Agentic AI\first app\venv\Scripts\python.exe" "c:\Users\nimishmishra\Desktop\Agentic AI\first app\src\Third day\Tool calling\llm-tool.py"

complete response from LLM

content="" additional\_kwargs={'refusal': None} response\_metadata={'token\_usage': {'completion\_tokens': 19, 'prompt\_tokens': 85, 'total\_tokens': 104, 'completion\_tokens\_details': {'accepted\_prediction\_tokens': 0, 'audio\_tokens': 0, 'reasoning\_tokens': 0, 'rejected\_prediction\_tokens': 0, 'text\_tokens': None}, 'prompt\_tokens\_details': {'audio\_tokens': 0, 'cache\_write\_tokens': None, 'cached\_tokens': 0, 'image\_tokens': None, 'text\_tokens': None}}, 'model\_provider': 'openai', 'model\_name': 'gpt-4o-mini-2024-07-18', 'system\_fingerprint': 'fp\_id2403c701', 'id': 'chatcmpl-EL1S1g0J3UHDElWjWvjeUe3HGmqQL', 'service\_tier': 'default', 'finish\_reason': 'tool\_calls', 'logprobs': None} id='lc\_run-01a07596-358c-73a3-8984-a6789df70356-0' tool\_calls=[{'name': 'multiply', 'args': {'a': 90854, 'b': 2345}, 'id': 'call\_rR0kv8C6WUA670nJA2e8b9sy', 'type': 'tool\_call'}] invalid\_tool\_calls=[] usage\_metadata={'input\_tokens': 85, 'output\_tokens': 19, 'total\_tokens': 104, 'input\_token\_details': {'audio': 0, 'cache\_read': 0}, 'output\_token\_details': {'audio': 0, 'reasoning': 0}}

=====

[{'name': 'multiply', 'args': {'a': 90854, 'b': 2345}, 'id': 'call\_rR0kv8C6WUA670nJA2e8b9sy', 'type': 'tool\_call'}]

\_\_\_\_\_tool details \_\_\_\_\_

multiply

{'a': 90854, 'b': 2345}

tools result

213052630

7 files and 0 cells to analyze

Ln 32, Col 24 Spaces: 4 UTF-8 CRLF Python first app (3.11.4)

When the LLM decides to call a tool, it often does not put the answer in response.content. Instead, it puts the tool request in response.tool\_calls. To achieve the response from the tool we need to use the tool-calling loop

```
tools = [current_time, multiply]

tools_by_name = {
    tool.name: tool
    for tool in tools
}

model = ChatOpenAI(
    model="gpt-4o-mini",
    temperature=0,
```

```

).bind_tools(tools)

messages = [
    SystemMessage(
        content="You are a helpful assistant. Use the tools when they
fit."
    ),
    HumanMessage(
        content=(
            "What time is it in Mumbai, and what is "
            "98765 times 43210?"
        )
    ),
]

# Tool-calling loop
step = 1
while True:
    # Ask the LLM
    response = model.invoke(messages)

    # Add the AI response to conversation history
    messages.append(response)

    # If no tool calls, we have the final answer
    if not response.tool_calls:
        print("\nFinal response:")
        print(response.content)
        break
    print(
        f"\nStep {step}: The model asked for "
        f"{len(response.tool_calls)} tool call(s)"
    )

    # Execute every tool requested by the LLM
    for call in response.tool_calls:

        # Get the requested tool
        tool_to_run = tools_by_name[call["name"]]

```

```

# Execute the tool with the arguments provided by the LLM
tool_message = tool_to_run.invoke(call)

print(
    f" {call['name']}({call['args']}) "
    f"-> {tool_message.content}"
)

# Add tool result back to the conversation
messages.append(tool_message)
step += 1

```

If you look at this code, you can see that we have to manually manage every step of the tool-calling process. This becomes difficult to maintain as the workflow gets more complex. To avoid this manual process, we can use an agent,

To avoid managing the entire tool-calling loop manually, we use an agent. The agent provides an abstraction layer that automatically handles the complete process deciding when a tool is needed, calling the appropriate tool, processing the tool result, and sending it back to the LLM when required. This removes the need for us to manually manage each step of the tool-calling workflow.

Here is the main difference in the flow of the both

Without Agent:

LLM → Check tool call → Execute tool → Send result to LLM → Repeat → Final response

↑

We manage everything manually

With Agent:

Agent

↓

LLM → Tool → Result → LLM → Tool → Result → Final Response

└──────── Agent manages the loop automatically ─────────┘

## Agent building in langchain :

Using the `create_agent` function, we can create an agent and register multiple tools with it. Each agent can also have its own system prompt, which defines the agent's role, responsibilities, and behavior.

```
agent = create_agent(  
    model="openai:gpt-4o-mini",  
    tools=[multiply, current_time],  
    system_prompt=(  
        "You are a helpful assistant. "  
        "Use the tools when they fit."  
    ),  
)
```

Now that the agent is ready, we can provide a user input and check the output. Unlike the manual tool-calling loop, we don't need to handle the tool calls ourselves. The agent automatically manages the tool-calling process and returns the final response.

```
result = agent.invoke(  
    {  
        "messages": [  
            {  
                "role": "user",  
                "content": (  
                    "What time is it in Mumbai, and what is 98765 times 43210?"  
                    "and when india was indepedent"  
                ),  
            },  
        ],  
    },  
)  
  
print("Final answer")  
print(result["messages"][-1].content) #last message in entire conversation
```

For the more example of agents : [Click here](#)

## Memory in agent :

By default, an agent only works with the messages provided to it for that invocation. Once the task is completed, the agent does not automatically remember the conversation.

A **checkpointer**, provided by LangGraph, can persist the conversation state using a unique `thread_id`. When the same `thread_id` is used again, the agent can continue the conversation from where it previously left off.

`InMemorySaver` is useful for learning and development, but it is **not suitable for production** because the stored memory is lost when the application restarts.

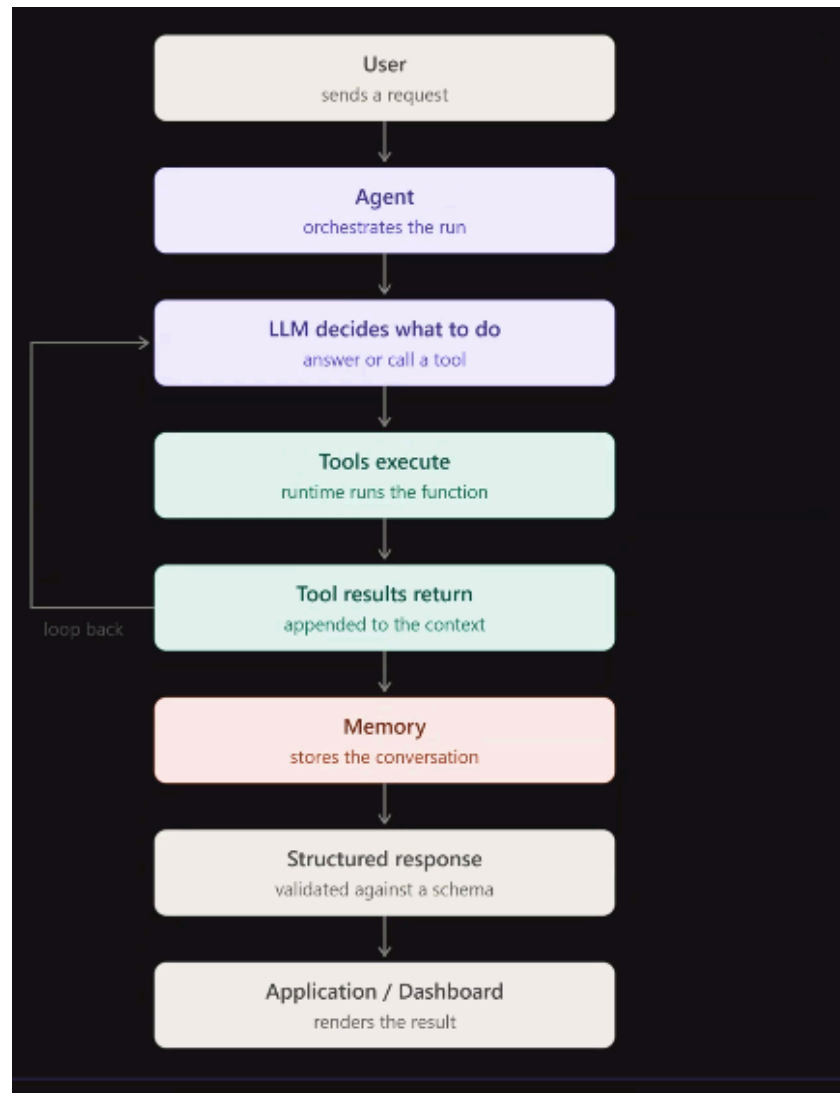
```
agent = create_agent(  
    model="openai:gpt-4o-mini",  
    tools=[book_seat],  
    system_prompt="You help learners book a seat in the LangChain class.",  
    checkpointer=InMemorySaver(), # keeps conversation memory  
)
```

Instead of `InMemorySaver` for production level app you can use any database

```
def ask(question, thread_id):  
  
    result = agent.invoke(  
        {  
            "messages": [  
                {  
                    "role": "user",  
                    "content": question  
                }  
            ],  
        },  
        config={  
            "configurable": {  
                "thread_id": thread_id  
            }  
        }  
    )  
  
    print(f"[{thread_id}] Q: {question}")  
    print(f"[{thread_id}] A: {result['messages'][-1].content}")  
  
ask("My name is Vinay", thread_id="vinay")  
  
ask("Book me a seat A12", thread_id="vinay")  
  
ask("waht is my name and what is my seat number ", thread_id="vinay")  
  
ask("waht is my name and what is my seat number ", thread_id="nimish") #here is has been changed
```

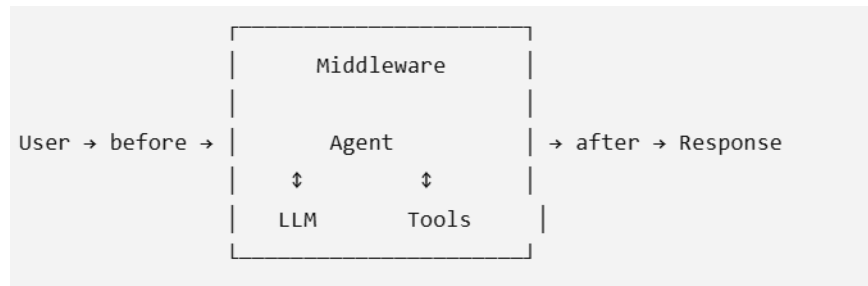
Here the thread id has the main role for memory management, at the basis `thread_id` it keeps the memory user wise.

Flow diagram for support project : [Link](#)



# Agent Middle ware

What is Agent Middleware?  
Think of your agent like this:

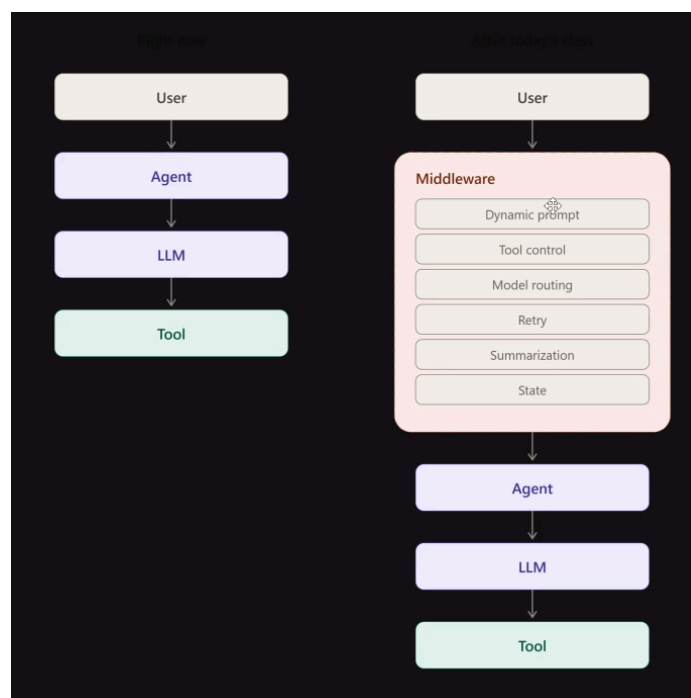


Think of **agent middleware as the security guard of an office.**

The office boss *could* check who is entering, verify permissions, and control access, but we don't want the boss to handle all these tasks. So, we have a security guard who handles these checks before allowing someone inside.

Similarly, **agent middleware sits between the user/agent and the execution process.** It can check requests, validate permissions, apply rules, modify inputs or outputs, and control what the agent is allowed to do—so the agent can focus on its main task.

With and without middleware



It gives you controlled interception points such as:

- **before\_agent**
- **before\_model**
- **after\_model**
- **after\_agent**
- **wrap\_model\_call**
- **wrap\_tool\_call**
- **dynamic\_prompt**

There are basically two styles.

#### **Node-style :**

These are good when you want to validate state, update state, add/remove messages, perform logging, enforce limits, terminate an agent early

Ex:

- **before\_model**: Executes before the model/LLM execution.  
Used to check or modify the state before sending the request to the model.
- **after\_model**: Executes after the model execution.  
Used to inspect, validate, or modify the model's response.
- **Before\_agent**
- **After\_agent**

#### **Wrap-style:**

These are good when you want to, modify the request, select another model, add tools, Retry, intercept errors, modify responses, skip the model call completely

Ex:

- **wrap\_model\_call**: Wraps the model execution.  
It can inspect/modify the request before the model runs and inspect/modify the response after the model runs.  
You control when/how handler(request) is called.
- **wrap\_tool\_call**: Wraps every tool execution.  
It can inspect the tool request before execution and the tool result/error after execution.

which will be executed first before model or wrap\_model\_call if both are being used ?

The execution flow will be like this:

before\_model → wrap\_model\_call → LLM → wrap\_model\_call → after\_model

Middleware can help control which tools are exposed to the LLM for a particular request. If an agent has 100 tools available, sending all 100 tools to the LLM on every request can increase token usage, latency, and potentially make tool selection harder. With middleware, we can inspect the request or current state and dynamically provide only the relevant subset of tools to the model.

Middleware can dynamically handle tools, models, agents and prompts.

Types of Middle ware :

- Built in Middleware
- Custom Middleware

### Built in Middle ware

| Middleware                | Purpose                                              |  |
|---------------------------|------------------------------------------------------|-------------------------------------------------------------------------------------|
| SummarizationMiddleware   | Summarizes conversation when it becomes too long     |                                                                                     |
| HumanInTheLoopMiddleware  | Pauses execution and asks a human for approval       |                                                                                     |
| ModelCallLimitMiddleware  | Limits how many times the model can be called        |                                                                                     |
| ToolCallLimitMiddleware   | Limits how many times tools can be called            |                                                                                     |
| ModelFallbackMiddleware   | Uses another model when the primary model fails      |                                                                                     |
| ModelRetryMiddleware      | Retries failed model calls                           |                                                                                     |
| LLMToolSelectorMiddleware | Uses an LLM to select relevant tools                 |                                                                                     |
| PIIMiddleware             | Detects/handles personally identifiable information  |                                                                                     |
| TodoListMiddleware        | Gives the agent todo-list/task-planning capabilities |                                                                                     |
| ContextEditingMiddleware  | Manages/edit conversation context                    |                                                                                     |
| ModelCallLimitMiddleware  | Controls model-call limits                           |                                                                                     |

All the Code of the Middle ware : [Click here](#)

Custom State in middleware :

**custom state** maintained by the agent/workflow to remember important information across the conversation. You can control the tool calling according to your requirements, not even just tools you can control, prompt, model everything using states.

Looks at this : [Click here](#)

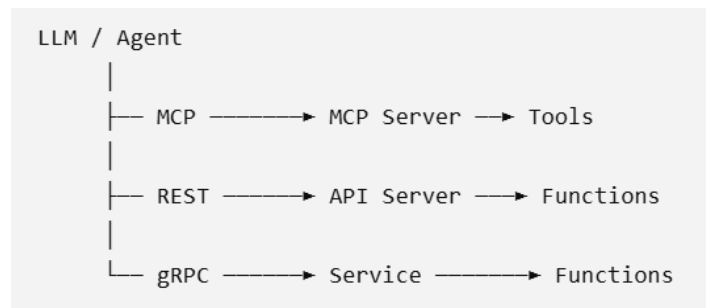
## MCP server

MCP (Model Context Protocol) server is a server that exposes tools, data, and resources to AI models through a standardized protocol, allowing agents to interact with external systems like databases, APIs, Gmail, or files so developers can use them.

It's just a protocol like HTTPs, GraphQL

| Protocol / Method     | How tool is called                                                        |
|-----------------------|---------------------------------------------------------------------------|
| MCP                   | Standardized tool calling between AI clients and MCP servers              |
| REST API              | HTTP request like <code>POST /search</code>                               |
| GraphQL               | Query/mutation to invoke backend operations                               |
| gRPC                  | Remote procedure call between services                                    |
| WebSocket             | Persistent connection for real-time tool interaction                      |
| Function/Tool Calling | LLM produces structured arguments; your application executes the function |

For an AI-agent architecture:



**So, if MCP is just a protocol, why do we need MCP when we already have REST APIs?**

Let's say we have APIs like: `/v1/create` → `/v2/create`(version changes)

### 1. API changes/versioning

If the API changes, how does an AI agent know which version to use and how to interact with it

The LLM does not automatically know your API changed. The MCP server is what tells(expose) the agent what tools are available and what inputs they need. So, when

your backend(MCP server side) moves from version 1 to version 2 Then the MCP client passes that updated schema to the LLM. So the LLM follows the new contract without needing to know the low-level API changes.

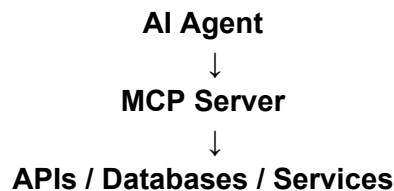
2. Payload/response changes

What if the request payload or response structure changes? The agent needs a standardized way to understand the tool's capabilities and expected inputs/outputs.

3. Different services

What if the functionality moves from one service to another? The underlying implementation may change, but the AI agent should not need to understand those internal changes.

That's where MCP comes in.



The MCP server exposes standardized tools and their schemas to the AI, while hiding the complexity of the underlying APIs and services.

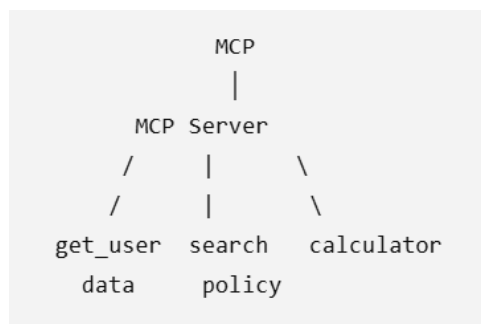
So, MCP doesn't replace REST APIs. It provides a standardized interface for AI applications to interact with those APIs, tools, and resources.

**Are all the MCP tools free ?** No sometimes it could be paid

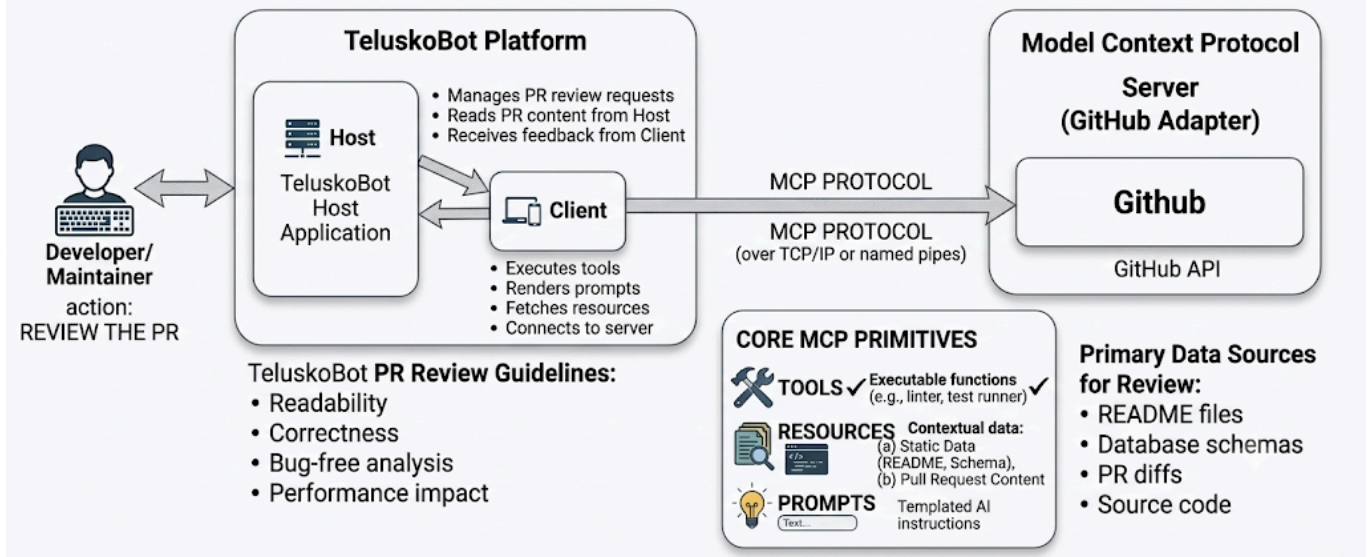
**A collection of tools can be known as the mcp?**

Not exactly, a collection of tools can be exposed through MCP, but the collection itself isn't necessarily called MCP.

Tools = individual capabilities exposed by that server.



# MCP (Model Context Protocol) Architecture - PR Review



Client here is MCP client

How will you create your own **MCP**?

we create our own MCP server, we can expose three main things to an LLM:

## 1. Tools

Tools are functions or actions that the LLM can execute through the MCP server. They are generally used when the AI needs to perform an operation or interact with an external system.

For example:

Calling a REST API

Creating, updating, or deleting an order

Querying a database

Sending an email

Searching customer information

The MCP server provides the tool name, description, and input schema so the LLM knows what the tool does and what parameters it requires.

## 2. Resources

Resources are information or data that the LLM can access through the MCP server. Unlike tools, resources are primarily used to read or retrieve information rather than perform an action.

For example:

- Reading a PDF or document
- Accessing files
- Reading database records
- Retrieving application configuration
- Accessing company knowledge or documentation

The MCP server tells the LLM what resources are available and how they can be accessed.

### **3. Prompts**

Prompts are predefined prompt templates or instructions exposed by the MCP server for specific tasks.

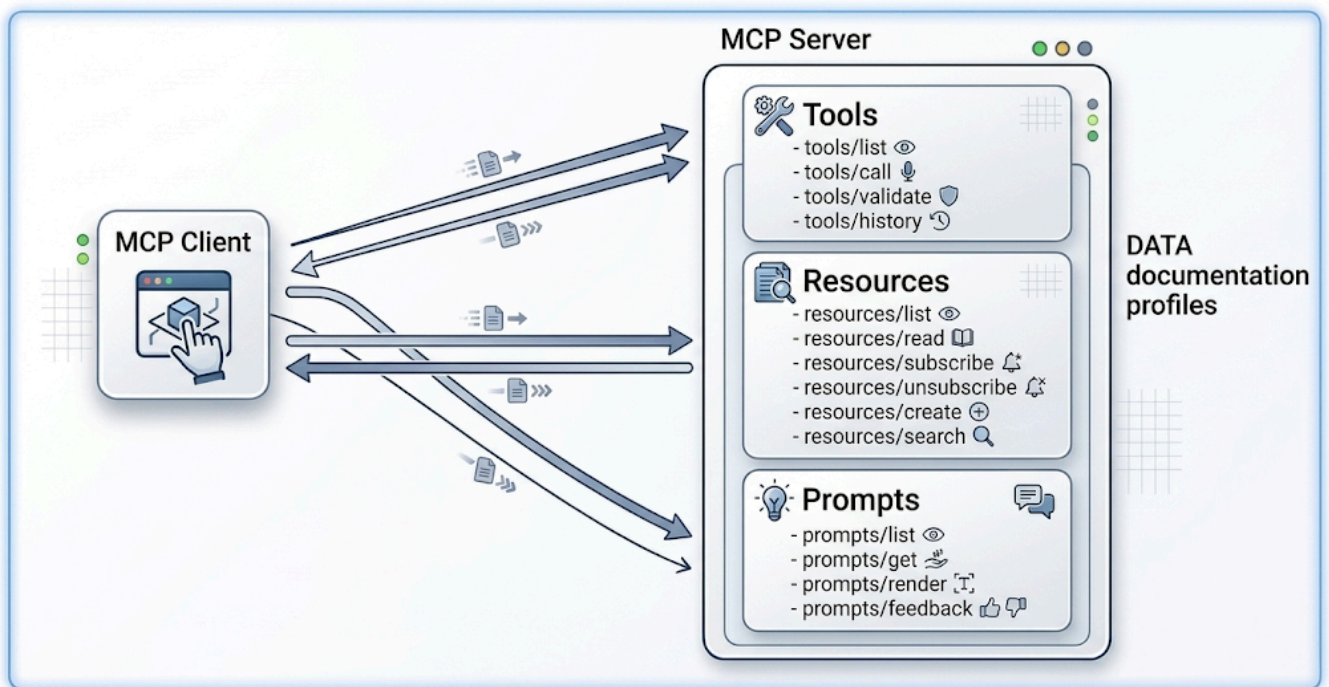
Instead of creating the entire instruction from scratch every time, an application can provide a reusable prompt.

For example:

- A customer-support prompt
- A code-review prompt
- A database-analysis prompt
- A document-summary prompt

The prompt can contain instructions and placeholders/arguments, which can be filled with the required information when the prompt is used.

So, our MCP server acts as a standardized bridge between the LLM and our external systems, exposing these three capabilities in a way the AI can understand and use.



- Here subscribe and unsubscribe are there if any change happens in resource then user get notify

MCP uses **JSON-RPC 2.0** as its messaging format and supports multiple transport mediums depending on whether the server is local or remote.

### JSON-RPC :

JSON-RPC stands for JSON Remote Procedure Call. It is a lightweight protocol that lets one application call a function/method on another application using JSON-formatted messages.

**JSON:** JavaScript Object Notation

**RPC:** A mechanism that allows you to call a function/procedure(Procedure) running on another machine(remote) or server as if it were a local function call.

How a JSON RPC req looks like

Request JSPONRPC :

```
{
  "jsonrpc": "2.0", //version you need to specify
  "method": "Add"
  "params" [2,4],
  "id": 1
}
```

Simple example

A client sends:

```
{
  "jsonrpc": "2.0",
  "method": "get_policy",
  "params": {
    "policy_id": "12345"
  },
  "id": 1
}
```

The server processes get\_policy and responds:

```
{
  "jsonrpc": "2.0",
  "result": {
    "policy_id": "12345",
    "status": "active"
  },
  "id": 1
}
```

If something goes wrong:

```
{
  "jsonrpc": "2.0",
  "error": {
    "code": -32601,
    "message": "Method not found"
  },
  "id": 1
}
```

There can be different types of errors, and the id is also included in the response so the client can identify which request the response belongs to.

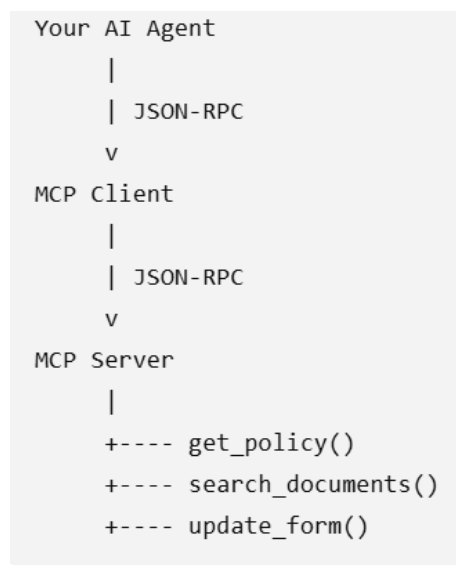
The important payload fields are:

| Field                | Purpose                                               |
|----------------------|-------------------------------------------------------|
| <code>jsonrpc</code> | Protocol version, normally <code>"2.0"</code>         |
| <code>method</code>  | Function/action you want to execute                   |
| <code>params</code>  | Arguments passed to the function                      |
| <code>id</code>      | Request identifier used to match request and response |
| <code>result</code>  | Successful response                                   |
| <code>error</code>   | Error response                                        |

RPC (Remote Procedure Call) allows a programmer to execute a function/procedure on another system as if it were a local function call.

### How it relates to MCP

Since you're learning MCP, JSON-RPC is particularly important because MCP uses JSON-RPC 2.0 for communication between the MCP client and MCP server.



The main difference is how you think about the operation.

**REST API** → you interact with resources

**RPC** → you call functions/methods

REST:

Simple example

Suppose you want to get a policy.

```
GET /policies/12345
```

Meaning:

“Give me the policy resource whose ID is 12345.”

RPC:

```
{
  "jsonrpc": "2.0",
  "method": "get_policy",
  "params": {
    "policy_id": "12345"
  },
  "id": 1
}
```

Meaning:

“Execute the get\_policy() function with policy ID 12345.”

RPC is especially useful in distributed applications because it makes communication between different services look similar to calling a local function.

## A. How to Call Tools Using JSON-RPC?

In MCP, tools are called using **JSON-RPC 2.0** messages. The client sends a `tools/call` request containing the tool name and its arguments. The MCP server executes the tool and returns the result in a JSON-RPC response.

### 1. Tool Call Request

The client sends a request to call the `search_tweets` tool:

```
{
  "jsonrpc": "2.0",
```

```

    "id": 2,
    "method": "tools/call",
    "params": {
      "name": "search_tweets",
      "arguments": {
        "query": "MCP",
        "limit": 5
      }
    }
  }
}

```

## 2. Tool Call Response

After executing the tool, the MCP server returns a response:

```

{
  "jsonrpc": "2.0",
  "id": 2,
  "result": {
    "content": [
      {
        "type": "text",
        "text": "Tweet 1: MCP is changing how AI connects to tools..."
      },
      {
        "type": "text",
        "text": "Tweet 2: Learning Model Context Protocol..."
      }
    ]
  }
}

```

The id remains 2 in the response so the client knows that this response belongs to the request with id: 2.

The actual tool output is returned inside the result object.

**Overall Flow: Client :** tools/call → MCP Server → Execute Tool → Return Result

## B. MCP Resources calling

In MCP, resources represent data or information that an MCP server can expose to a client. Unlike tools, which are used to perform an action, resources are mainly used to provide or monitor data.

For example, an MCP server might expose:

- A file
- A database record
- A configuration
- Documentation
- A live application state
- A log
- A dynamically changing piece of information

There are three important resource operations to understand:

1. **Resources/read** : The resources/read operation is used when the client wants to retrieve the current content of a resource from the MCP server.

Req :

```
{
  "jsonrpc": "2.0",
  "id": 10,
  "method": "resources/read",
  "params": {
    "uri": "file:///documents/policy.txt"
  }
}
```

Response :

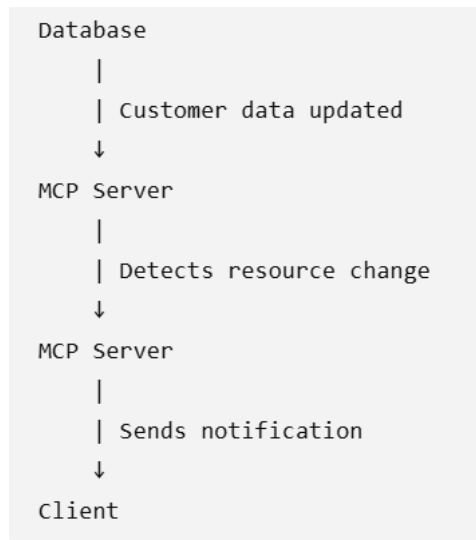
```
{
  "jsonrpc": "2.0",
  "id": 10,
  "result": {
    "contents": [
      {
        "uri": "file:///documents/policy.txt",
        "mimeType": "text/plain",
        "text": "This is the policy document."
      }
    ]
  }
}
```

2. **Resources/subscribe** : With resources/read, the client asks "Give me the resource now." With **resources/subscribe** : the client says: "Keep watching this resource and notify me if it changes." For example, imagine an MCP server exposes

Req :

```
{
  "jsonrpc": "2.0",
  "id": 11,
  "method": "resources/subscribe",
  "params": {
    "uri": "database://customer/123"
  }
}
```

Now the important part happens.  
Suppose later the customer data changes.



The client can receive a notification such as(res) :

```
{
  "jsonrpc": "2.0",
  "method": "notifications/resources/updated",
  "params": {
    "uri": "database://customer/123"
  }
}
```

There is no id in the notification. That is because this is a notification, not a request that expects a response.

3. **Resources/unsubscribe** → Stop Watching a Resource After subscribing, the client may no longer need updates.

```
{
  "jsonrpc": "2.0",
  "id": 12,
  "method": "resources/unsubscribe",
  "params": {
    "uri": "database://customer/123"
  }
}
```

Resources are **data that the MCP server makes available** to the client and **tools** are **functions that the MCP server can execute** on behalf of the client.

when response having id it means its catering a single request but if it don't have id means it's just a notification that doesn't belong to any single request

- **Request with id** : Expects a response associated with that specific request.

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "method": "tools/call",
  "params": {
    "name": "search_tweets"
  }
}
```

- **Response with id** : Identifies which request the response belongs to.

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "result": {
    "tweets": []
  }
}
```

- **Notification without id** : Does not expect a response and is not associated with a request/response cycle.

```
{
  "jsonrpc": "2.0",
  "method": "notifications/resources/updated",
  "params": {
    "uri": "file:///document.txt"
  }
}
```

### C. MCP Prompt calling :

The prompts/get method is used by the MCP client to retrieve a specific prompt template from the MCP server. The client provides the prompt name and optional arguments, and the server returns the generated prompt messages that can be sent to the LLM.

Req :

```
{
  "jsonrpc": "2.0",
  "id": 3,
  "method": "prompts/get",
  "params": {
    "name": "search_tweets",
    "arguments": {
      "query": "MCP",
      "limit": 5
    }
  }
}
```

Res:

```
{
  "jsonrpc": "2.0",
  "id": 3,
  "result": {
    "description": "Search for tweets",
    "messages": [
      {
        "role": "user",
        "content": {
          "type": "text",
          "text": "Search for tweets"
        }
      }
    ]
  }
}
```

## Batching in JSONRPC:

In JSON-RPC 2.0, you can send multiple requests together in a single HTTP/message payload. This is called batching.

### Without batching

You send requests separately:



Multiple requests are placed inside a **single JSON array**:

```
[
  {
    "jsonrpc": "2.0",
    "id": 1,
    "method": "tools/list"
  },
  {
    "jsonrpc": "2.0",
    "id": 2,
    "method": "prompts/list"
  },
  {
    "jsonrpc": "2.0",
    "id": 3,
    "method": "resources/list"
  }
]
```

## Benefits of JSON-RPC

1. **Lightweight & Simple**

JSON-RPC uses a simple JSON-based request/response structure, making it easy to implement, understand, and integrate.

2. **Transport Independent**

JSON-RPC is not tightly coupled to a specific transport mechanism. It can work over **HTTP, WebSockets, TCP, stdio**, or even custom transports.

This makes it suitable for different distributed-system architectures.

3. **Supports Bidirectional Communication**

Unlike traditional request-only REST APIs, protocols built on JSON-RPC can support communication **in both directions**—the client can send requests, and the server can also send messages such as notifications when appropriate.

4. **Batching**

Multiple JSON-RPC requests can be sent together as a **single batch**, and the server can return multiple responses together. This can reduce communication overhead.

**Multiple Requests → One Batch → Multiple Responses**

5. **Notifications**

A client can send a message **without expecting a response**, and the server can also send notifications when supported by the surrounding protocol.

For example, in MCP, a server can notify the client that a **resource has been updated**, allowing the client to react without repeatedly polling the server.

## Transport Layer

The **transport layer** defines how the client and server communicate and exchange JSON-RPC messages.

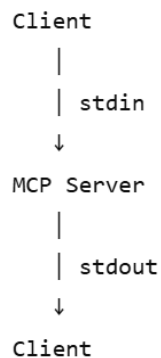
For MCP, common transport mechanisms include **stdio** and **HTTP-based transports**.

### 1. STDIO - Local Communication

**STDIO (Standard Input/Output)** is primarily used when the MCP server runs **locally on the same machine as the client**.

It uses the operating system's standard streams:

- **stdin** → Client sends messages to the MCP server
- **stdout** → MCP server sends messages back to the client



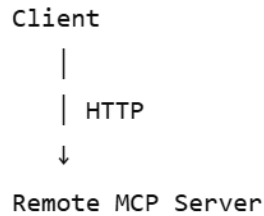
### Why STDIO?

- Fast — no network communication is required
- Simple — no port or HTTP server needs to be exposed
- Lightweight
- Suitable for local MCP servers
- Can provide a smaller network attack surface because communication stays through the local process streams

**Important:** STDIO is primarily suited to local process communication. For a remotely hosted MCP server, an HTTP-based transport is generally used.

## 2. HTTP-Based Transport

When the MCP server is running on a **remote machine**, communication generally happens over a network using an **HTTP-based transport**.



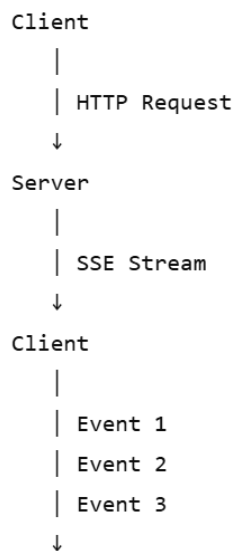
HTTP provides:

- Network-based communication
- Remote server access
- Compatibility with standard web infrastructure
- Support for authentication, proxies, load balancers, etc.

### HTTP + Server-Sent Events (SSE)

**SSE (Server-Sent Events)** is a mechanism built on top of HTTP that allows a server to **continuously send events to a connected client**.

This is useful when the server needs to send updates asynchronously instead of requiring the client to repeatedly make requests.



## Simple Comparison

| Transport  | Typical Use                       | Communication         |
|------------|-----------------------------------|-----------------------|
| STDIO      | Local MCP server                  | Process streams       |
| HTTP       | Remote MCP server                 | Network communication |
| HTTP + SSE | Streaming server-to-client events | HTTP + event stream   |

### In simple terms:

STDIO : Best suited for local process communication

HTTP : Suitable for remote/network communication

SSE : Enables server-to-client event streaming over HTTP

### MCP life cycle :

#### 1. Phase 1: Initialization Phase

The initialization phase establishes protocol compatibility and negotiates capabilities. Neither party can process functional requests until this handshake finishes.

- **initialize request:** Sent from Client to Server. Contains the supported protocol version, client implementation details, and supported client capabilities (e.g., filesystem roots, sampling support).
- **initialize response:** Sent from Server to Client. The server verifies compatibility and responds with its details, negotiated protocol version, and server capabilities (e.g., available tools, resources, prompt templates, or logging).
- **initialized notification:** Sent from Client to Server. An unacknowledged JSON-RPC notification sent by the client confirming that initialization is complete and operational traffic can begin.

#### 2. Phase 2: Operation Phase

Once the handshake is completed, the connection transitions into normal protocol operations:

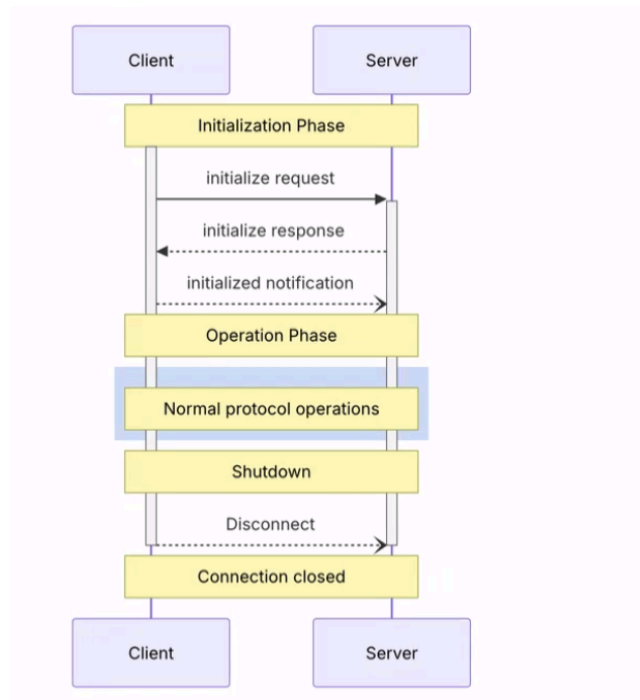
- **Tool & Resource Discovery:** The client lists available tools, prompts, or resources declared by the server.

- **Execution & Data Exchange:** The LLM executes tool calls, reads context resources, or subscribes to dynamic updates via JSON-RPC messages.
- **Logging & Ping:** Background operations such as health checks (pings) and server-side log streaming occur during this state.

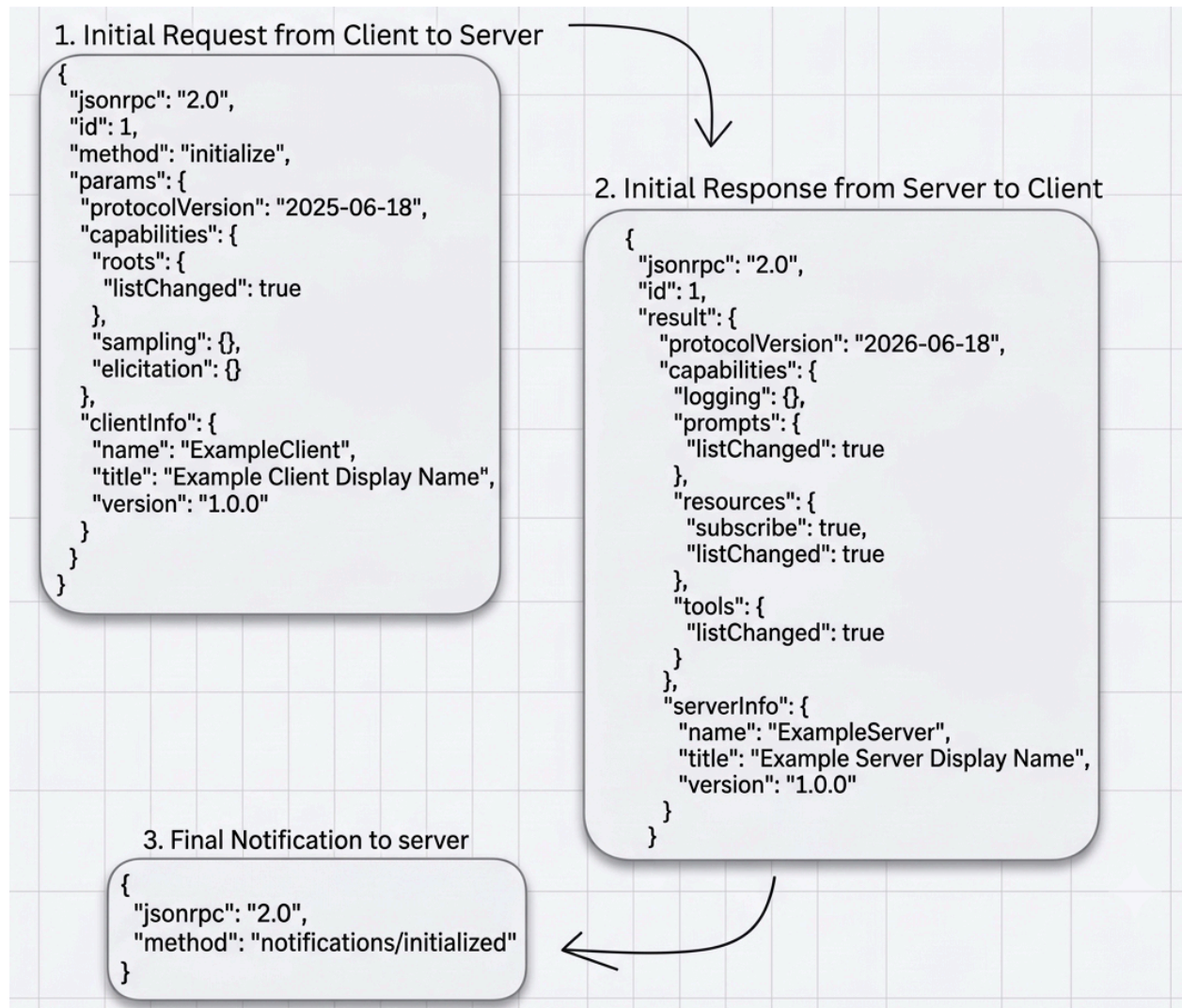
### 3. Phase 3: Shutdown Phase

The shutdown phase ensures graceful termination and resource cleanup:

- **Disconnect:** Initiated when the client or host closes the transport layer (e.g., closing input streams for local stdio connections or terminating SSE HTTP sessions).
- **Connection closed:** The process terminates cleanly, releasing system resources, database connections, and memory state on both ends.



# MCP Life Cycle



## 1st Stage: Initialization

### Step 1: Client starts the conversation

The **MCP client initiates the connection** by sending an initialize request containing:

- Protocol version
- Client capabilities
- Client information: name and version

It looks like this:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "initialize",
  "params": {
    "protocolVersion": "2025-06-18",
    "capabilities": {
      "roots": {
        "listChanged": true
      },
      "sampling": {},
      "elicitation": {}
    },
    "clientInfo": {
      "name": "ExampleClient",
      "title": "Example Client Display Name",
      "version": "1.0.0"
    }
  }
}
```

## Capabilities

### Roots:

Provides the server with information about the roots/directories that the client makes available. For example, allowing an MCP client to expose specific project directories.

### Sampling:

The server can request the **client to perform an LLM sampling/generation operation**. For example, the MCP client may have access to an LLM, and the server can request the client to use it.

### Elicitation:

The server can request additional information from the user through the client when required.

For example, suppose we are using a Blinkit MCP and a product becomes unavailable during an order. The MCP server may need additional information or confirmation from the user.

So, **elicitation comes into the picture when the server needs information or confirmation that is not currently available**.

## Step 2: Server responds

The **MCP server responds** with:

- Protocol version
- Server capabilities
- Server information: name and version
- Optional instructions

Response looks like this:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "protocolVersion": "2025-06-18",
    "capabilities": {
      "logging": {},
      "prompts": {
        "listChanged": true
      },
      "resources": {
        "subscribe": true,
        "listChanged": true
      },
      "tools": {
        "listChanged": true
      }
    },
    "serverInfo": {
      "name": "ExampleServer",
      "title": "Example Server Display Name",
      "version": "1.0.0"
    },
    "instructions": "Optional instructions the client"
  }
}
```

The server **does not send the complete list of tools, resources, or prompts** at this stage. This is only the introduction/handshake between the client and server.

## Server Capabilities

**Tools:** Actions/functions that the client can call.

**Resources:** Data or information that the client can read or subscribe to.

**Prompts:** Reusable prompt templates provided by the server.

**Logging:** Server log messages that can be sent to the client and streamed/displayed.

### Step 3: Client sends initialization notification

The handshake is completed when the client accepts the server's response and sends an notification/initialized notification as an acknowledgement.

It looks like this:

```
{
  "jsonrpc": "2.0",
  "method": "notifications/initialized"
}
```

If the client and server cannot agree on a compatible protocol version, initialization fails, so the normal operation phase does not begin.

## 2nd Stage: Operation Phase

The **MCP client now uses the capabilities provided by the MCP server.**

At this stage, the client can request the lists of:

- Tools
- Resources
- Prompts

The client can then call a tool, read a resource, get a prompt, subscribe to resource updates, etc.

For example:

```
Client
|
|---- tools/list -----> Server
|<--- Tool List -----|
|
|---- tools/call -----> Server
|<--- Tool Result -----|
```

## 3rd Stage: Shutdown

The **shutdown is generally initiated by the client/application.**

- No specific JSON-RPC shutdown message is required.
- The shutdown is handled by the underlying **transport layer**.

## STDIO

Use STDIO when the MCP server runs on the same machine as the client. The client/server closes the streams:

```
Input Stream → Close  
Output Stream → Close
```

## HTTP

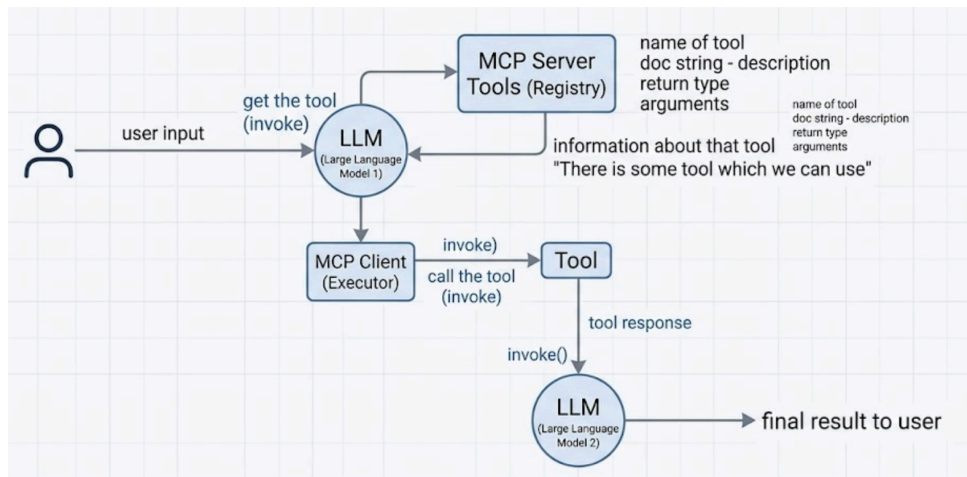
Use HTTP when the MCP server is running separately from the client, especially when it is deployed on a server/cloud. The HTTP connection/session is closed according to the HTTP transport mechanism.

Click here : [Demo](#)

```
Client  
|  
| Close connection  
↓  
Server
```

The user query comes first. The MCP Client retrieves the available tools from the MCP Server and provides them to the LLM along with any internal tools.

The LLM evaluates which tool is required, and if an MCP tool is needed, it instructs the MCP Client to call that tool. The MCP Client sends the request to the MCP Server and receives the tool result. The result can then be passed back to the LLM to generate a better, more contextual final response or returned directly to the user if no further LLM processing is required.



When the LLM needs to call an MCP tool, it first examines the tool's **metadata**, including its **description**, **function signature**, **required arguments**, and return type.

To execute an MCP tool, the LLM relies on an **MCP Client** running on the application side. There are two important interactions with the MCP Server:

#### 1st interaction - **Tool Discovery**

```
tools = await client.getTools()
```

Here, the MCP Client connects to the MCP Server and retrieves the available tools and their metadata.

The connection/session is established as part of the MCP client-server communication.

#### 2nd interaction - **Tool Execution**

```
result = await tool.invoke(tool_call["args"])
```

When the LLM selects an MCP tool, the MCP Client invokes that tool with the required arguments. The request is sent to the MCP Server, the server executes the tool, and the result is returned to the client.

**session in MCP** : MCP client creates/opens a connection → MCP server associates that connection with a session → multiple MCP messages use that session → session ends when the connection is closed.

A session is **not the same as a conversation memory**. If you want data to survive reconnections, you need external persistence such as Redis or a database.

Click here : [Demo](#)

## What is an MCP notification?

In MCP, communication is **not only a request → response**.  
There are two important types of messages:

A notification is a **one-way message**:

```
Server — Notification —> Client
```

There is no response expected from the client.

For example, while the server is doing some long-running work:

```
Server
|
| "Step 1 completed"
▼
Client
```

The client doesn't need to respond with:

```
"Okay, I received it."
```

**A handle that gives the running tool access to the current MCP communication/session.**

Just look at this example, how a server's tool accessing a client function and notifying.

Demo : [Link](#)

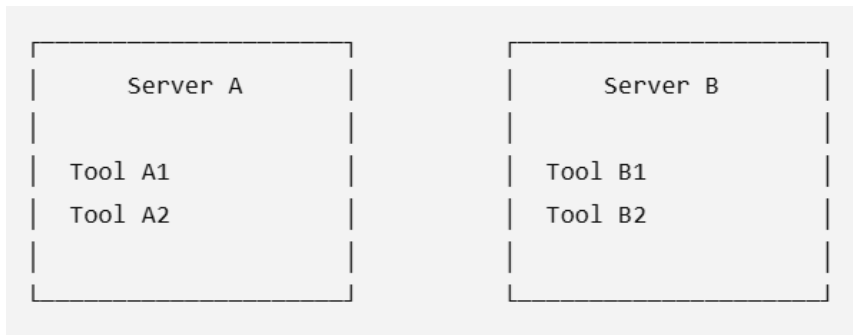
### Calling a tool inside another tool :

lets say we have 2 server A and B I want to use tool of B in A then we will use B server instance as client (using client) then we will call that tool

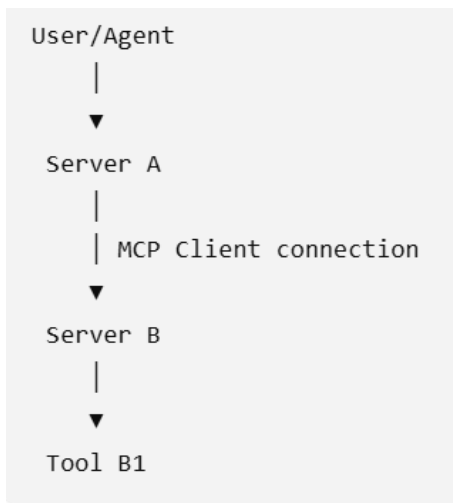
ex: async with Client(url) as http :

http.tool\_Name like this

You have two independent MCP servers:



Server A needs to connect to Server B as an **MCP client**:



Demo : [Link](#)

## Error Handling in MCP

In FastMCP, we can use `mask_error_details=True` to prevent exposing the actual internal error details to the client/user.

When an error occurs, FastMCP returns a masked/generic error message instead of exposing sensitive implementation details such as stack traces, internal paths, database information, or other debugging information.

```
mcp = FastMCP(  
    "My Server",  
    mask_error_details=True  
)
```

For example, if the actual error is:

```
Database connection failed:  
Server=internal-db-01; Password=xxx; Connection timeout...
```

But with the `mask_error_details` user will receive a **masked error** instead of the complete internal error details.

The `env` parameter can be used to control how much logging the MCP server produces. The log level determines **which messages are shown** based on their severity.

The available levels are:

- `DEBUG` → Most detailed logs; useful for development/debugging.
- `INFO` → Normal application/server information.
- `WARNING` → Something unexpected but not necessarily an error.
- `ERROR` → Something failed.
- `CRITICAL` → Serious failure requiring immediate attention.

```
"stdio_server": {  
    "transport": "stdio",  
    "command": sys.executable,  
    "args": [str(SERVER_PATH)],  
    "env": {  
        "FASTMCP_LOG_LEVEL": "WARNING"  
    }  
}
```

When we call an MCP tool, the tool returns a result that contains an `isError` field. **`isError`** indicates whether the tool execution resulted in an error:

```
result = await session.call_tool(tool_name, arguments)

tool_output = result.content[0].text
if result.isError:
    print(" ERROR ->", tool_output)
else:
    print(" OK ->", tool_output)
```

Demo : [Link](#)

## How Authentication works with MCP:

Securing an MCP Server with JWT

To secure an MCP server, we can use JWT (JSON Web Token) authentication.

The MCP server can be protected by configuring the `auth` parameter with a `JWTVerifier`:

```
mcp = FastMCP(
    "Production-style Secure Server",
    auth=JWTVerifier(
        public_key=key_pair.public_key,
        issuer=ISSUER,
        audience=AUDIENCE,
    )
)
```

How it works

- The client sends a JWT token when accessing the MCP server.
- The server validates the token using the configured `public_key`.
- It also verifies values such as the issuer and audience.
- If the token is invalid, expired, or does not match the expected configuration, the MCP server will reject the request.
- Therefore, only clients with a valid and trusted token can access the MCP server.

## Role/Scope-Based Tool Security

We can secure individual tools using scopes on the basis of user roles.

For example:

```
@mcp.tool(auth=require_scopes("db:reset"))
def reset_database() -> str:
    """Reset the database. Dangerous - needs the db:reset scope!"""
    print("[server] reset_database was called!")
    return "Database reset done."
```

Here, db:reset is the required scope.

The flow becomes:

**JWT Authentication → Token Validation → Role Validation → Tool Access(db:reset)**

So, even if the client has a valid JWT token, it cannot call reset\_database() unless the token contains the required db:reset scope.

Authentication answers: "Who are you?"

Authorization/Scopes answer: "What are you allowed to do?"

HMAC is used to verify that the request was generated by someone who possesses the **shared secret** and that the message has not been modified.

Demo : [Link](#)

## GIT hub MCP server project

Demo : [Link](#)

## Pydantic in Python

Pydantic is a Python library used to define and validate structured data using rules. Think of Pydantic as a rule book or contract that defines what data is allowed.

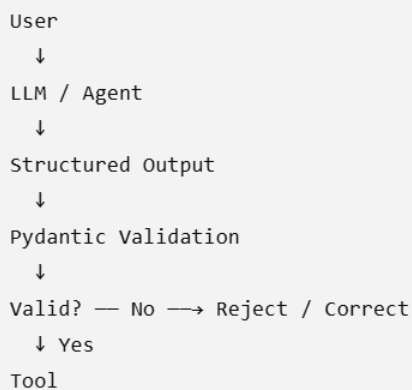
For example:

- If a field is an amount, we can define that it must be a valid number and follow specific constraints.
- If a field is currency, we can restrict it to values such as INR, USD, or EUR.
- We can also define whether a field is required, its data type, minimum/maximum values, allowed options, etc.

This becomes especially important when working with LLMs.

We should not blindly trust the output generated by an LLM. If an LLM generates parameters that are passed to a tool, those parameters should first be validated against our predefined schema.

LLM → Pydantic Validation → Tool



For example, if an LLM generates:

```
{
  "amount": 100,
  "currency": "INR"
}
```

Pydantic validates that this data matches the expected structure and rules **before it reaches the tool**.

```
from typing import Literal
from pydantic import BaseModel, Field
from langchain.tools import tool

class PaymentRequest(BaseModel):
    amount: float = Field(gt=0, le=100000)
    currency: Literal["INR", "USD", "EUR"]

@tool(args_schema=PaymentRequest)
def make_payment(amount: float, currency: str) -> str:
    """Make a payment using the specified amount and currency."""
    return f"Payment of {amount} {currency} initiated."
```